

Grundlagen

Diskretisierung

Definitionsbereich

Der Definitionsbereich der Funktion wird auf das Intervall $[-3, 1]$ begrenzt und in Schritte von 0.2 aufgelöst

Fehleranalyse

In der Numerik wird die Bewertung anhand folgender Kriterien unterschieden:

Kondition

Hängt von der Genauigkeit ab, mit der man die Zahlenwerte (z.B. 0, 1, 2, $\sqrt{5}$, π) abbilden können

Python arbeitet mit 64-bit Gleitkommazahlen, d.h. die Zahlen sind oft bis auf die 15-17te Kommastelle genau

Stabilität

Zeigt an, wie viel Genauigkeit man durch die Darstellung von reellen Zahlen als Gleit- oder Festkommazahl verliert

Konsistenz

Hängt von der Schrittweite bzw. Abtastrate ab, die man dem Algorithmus vorgibt

Brute Force

1. Gegeben ist Funktion $f: \mathbb{R} \rightarrow \mathbb{R}$
2. Bestimme Suchintervall $[a; b]$
3. Wähle Schrittweite $h > 0$ (z.B. $h = 0,01$)
4. Durchsuche alle Punkte
 - ↳ Für alle $x_i = a + i \cdot h$ mit $i = 0, 1, 2, \dots$, solange $x_i \leq b$
 - ↳ Berechne $f(x_i)$
 - ↳ Speichere $x_{\min}$ falls $f(x_i) < f(x_{\min})$

Newtonverfahren

Verfahren findet Nullstellen einer gegebenen eindimensionalen Funktion

1. Wir beginnen bei einem Startpunkt x_0
2. Anlegen einer Tangente an x_0
3. x-Wert des Nullpunktes der Tangente ist neuer x-Wert für unsere Funktion
4. Solange die Genauigkeit noch nicht erreicht ist, zurück zu Schritt 2 mit neuem x-Wert

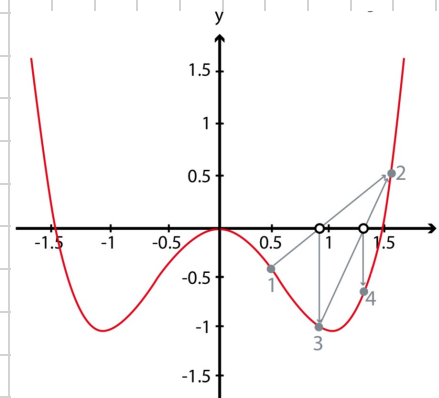
Falls die Ableitung nicht analytisch lösbar ist, mit Hilfe von Steigungsdreieck numerisch lösen

! Newtonverfahren versagt, wenn f' beim Startwert 0 annimmt

Sekantenverfahren

Grundidee ähnelt dem Newtonverfahren, allerdings wird diesmal mit 2 Punkten und dessen Verbindung gearbeitet

$$x_{n+1} = x_n - \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \cdot f(x_n)$$



Performancekennzahlen

Zeitkomplexität

Wie stark skaliert die Laufzeit mit der Menge der Eingabedaten?

Konvergenzgeschwindigkeit

Wie schnell erhöht sich die Genauigkeit pro Iterationsschritt?

Laufzeit einer Iteration

Wie lange wird für einen Durchlauf benötigt?

Tatsächliche Laufzeit

Wie lange benötigt der Algorithmus bis zur Konvergenz?

Formel für die Konvergenzordnung

$$\lim_{k \rightarrow \infty} \sup \frac{|x_k - x|}{|x_{k-1} - x|^q} = \mu$$

Suche nach Extremstellen

	Newtonverfahren	Sekantenverfahren
Nullstellen	$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$	$x_{n+1} = x_n - \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} f(x_n)$
Extremstellen	$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$	$x_{n+1} = x_n - \frac{x_n - x_{n-1}}{f'(x_n) - f'(x_{n-1})} f'(x_n)$

Suche nach Extremstellen für Skalarfelder

	Newtonverfahren	Wir benötigen ...
Nullstellen	$\vec{x}_{n+1} = \vec{x}_n - \frac{f(\vec{x})}{ \nabla f ^2} \nabla f$	$\nabla f = \left(-\frac{\partial f}{\partial x_1} \quad -\frac{\partial f}{\partial x_2} \quad \dots \right)^T$
Extremstellen	$\vec{x}_{n+1} = \vec{x}_n - H^{-1} \cdot \nabla f$	$H = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_1 \partial x_3} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots \\ \frac{\partial^2 f}{\partial x_3 \partial x_1} & \dots & \dots \end{pmatrix}$

Berechnung der Hesse-Matrix schwierig, deswegen wird oft auf Gradient Descent zurückgegriffen

Differenzialgleichungen

Eulerverfahren

Gegeben ist eine DGL $f(x)$ mit einem AWP $P(x_0 | f(x_0))$

1. Anzahl der Schritte und Schrittweite $h > 0$ festlegen
2. Bis zum nächsten Punkt $x_0 + h$ rechnen wir mit der Steigung am Punkt $f(x_0)$
3. Dort angekommen erhalten wir einen neuen Wert für y und somit auch für y'
4. Zurück zu Schritt 2 und für die festgelegte Schrittweite wiederholen

Runge-Kutta-Verfahren

Weiterentwicklung des Eulerverfahrens, um Fehler zu minimieren.

Diesmal nimmt man nicht nur die Steigung zu Beginn des Schrittes, sondern auch die in der Mitte und am Ende

Runge-Kutta 4

$$y_{n+1} = y_n + \frac{h}{6} \cdot (k_1 + 2k_2 + 2k_3 + k_4)$$

$$t_{n+1} = t_n + h$$

$$k_1 = f(y_n, t_n)$$

$$k_2 = f\left(y_n + \frac{hk_1}{2}, t_n + \frac{h}{2}\right)$$

$$k_3 = f\left(y_n + \frac{hk_2}{2}, t_n + \frac{h}{2}\right)$$

$$k_4 = f(y_n + hk_3, t_n + h)$$

Butcher-Table

	Schrittweite für y			
Schrittweite für t	0			
	$\frac{1}{2}$	$\frac{1}{2}$		
	$\frac{1}{2}$	0	$\frac{1}{2}$	
	1	0	0	1
		$\frac{1}{6}$	$\frac{2}{6}$	$\frac{2}{6}$
		$\frac{1}{6}$	$\frac{2}{6}$	$\frac{1}{6}$
		$\frac{1}{6}$	$\frac{2}{6}$	$\frac{1}{6}$
		$\frac{1}{6}$	$\frac{2}{6}$	$\frac{1}{6}$

Gewichtung im Iterationsschritt

Runge-Kutta 4

$$y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

$$t_{n+1} = t_n + h$$

$$k_1 = f(y_n, t_n)$$

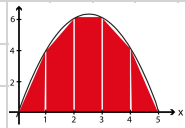
$$k_2 = f\left(y_n + \frac{hk_1}{2}, t_n + \frac{h}{2}\right)$$

$$k_3 = f\left(y_n + \frac{hk_2}{2}, t_n + \frac{h}{2}\right)$$

$$k_4 = f(y_n + hk_3, t_n + h)$$

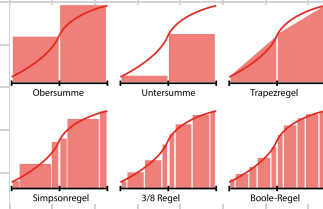
Numerische Integration

Trapezregel



$$\begin{aligned} A_1 &= 0.5 \cdot (0+4) \cdot 1 = 2 \\ A_2 &= 0.5 \cdot (4+6) \cdot 1 = 5 \\ A_3 &= 0.5 \cdot (6+6) \cdot 1 = 6 \\ A_4 &= 0.5 \cdot (6+4) \cdot 1 = 5 \\ A_5 &= 0.5 \cdot (4+0) \cdot 1 = 2 \end{aligned} \quad A = 20$$

Newton - Cotes - Formeln



Gewichtung

	Stützstelle	0	1
	Gewichtung	1/2	1/2

	Stützstelle	0	1/2	1
	Gewichtung	1/6	4/6	1/6

	Stützstelle	0	1/3	2/3	1
	Gewichtung	1/8	3/8	3/8	1/8

	Stützstelle	0	1/4	1/2	3/4	1
	Gewichtung	7/90	32/90	12/90	32/90	7/90

Formel Trapez

$$A = \frac{1}{2} \cdot h \cdot (a + c)$$

a, c - Länge der parallelen Seiten